为什么你的龙虾这么慢

对于这个问题，我们想回答的不是“某个函数慢了多少毫秒”，而是一个更实际的问题：为什么同一套网关，在单 worker 时表现稳定，一旦变成多 worker、多会话并发，整体响应时间就会上升一个数量级。

先说结论

- 同样都是 500 次请求，单 worker 场景的总耗时均值是 `1061ms`，10 个 worker 并发后变成 `10161ms`，慢了约 `10` 倍。
- 这次变慢，主要发生在“等待执行完成（`wait`）”阶段，而不是“读取历史结果”阶段。
- 真正拖慢系统的，不是模型推理、CPU 打满或磁盘裸读写变慢，而是“会话数量变多带来的重复冷启动”叠加“会话状态管理链路在并发下更拥堵”。

哪些场景会遇到这个问题

这个问题并不只出现在“多 agent”这个词上，它更广泛地出现在“同一个网关同时服务多个独立执行单元”的场景里。

多 agent 的典型场景是：一个主控把任务拆给多个 worker，并行查资料、读代码、跑工具、汇总结果。表面上看是一项任务，实际上背后是多个执行单元同时向同一个网关发起请求。

多 session 的典型场景是：多人同时接同一个服务、一个人同时开多个标签页或多个项目、批量自动化任务一起发起请求。表面上不一定叫“多 agent”，但本质上也是多个会话同时在争抢同一套后端资源。

这次测试更接近后一种现实使用形态：同一个网关同时服务多个 worker、多个会话。它不是“单条请求内部再拆 10 个 agent”的严格模拟，但足够贴近外部并发使用时的压力模型，因此对实际体验具有较强参考意义。

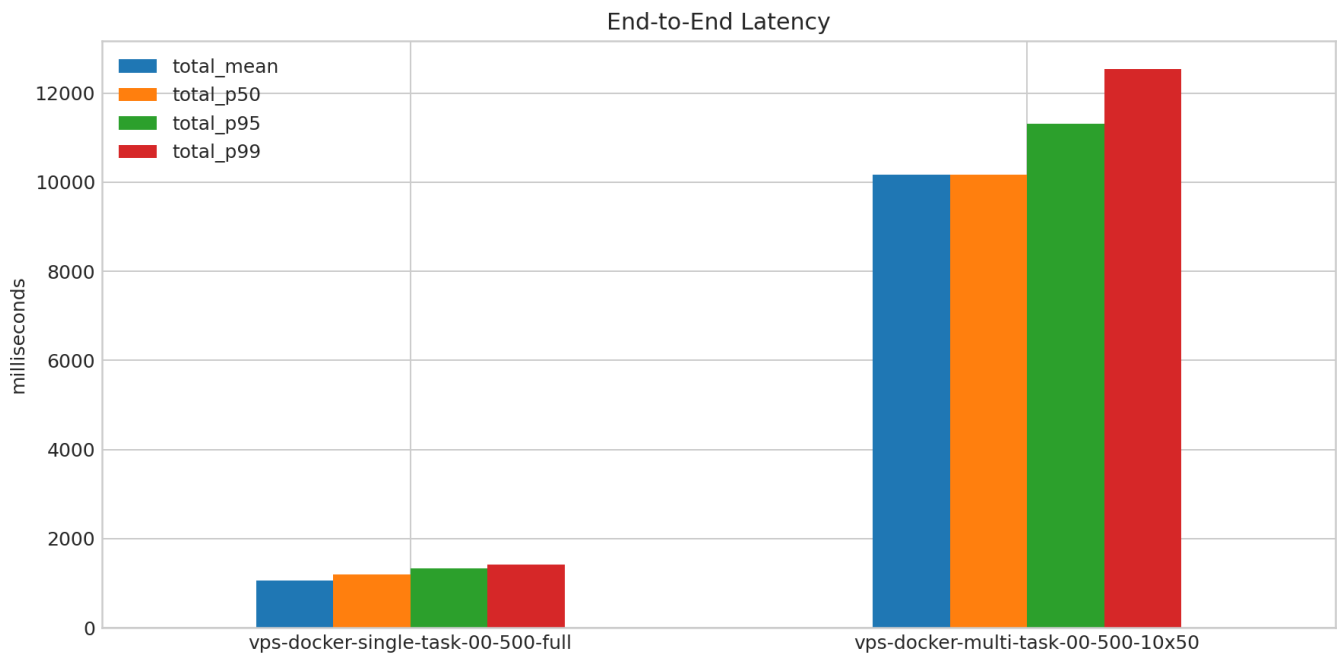
这次实验怎么做

场景	并发数	每个 worker 请求数	总请求数	会话策略	启动间隔	消息内容	运行环境	采集项	成功率
单 worker 基线	1	500	500	每个 worker 一个会话（`per_worker`）	0ms	`/context list`	Docker + `openclaw:bench-local` + openEuler 24.03 LTS + Python 3.12.10	docker stats、pidstat、strace、node trace、iostat、perf、vmstat	500/500

场景	并发数	每个 worker 请求数	总请求数	会话策略	启动间隔	消息内容	运行环境	采集项	成功率
10 worker 并发	10	50	500	每个 worker 一个会话（`per_worker`）	150ms	`/context list`	Docker + `openclaw:bench-local` + openEuler 24.03 LTS + Python 3.12.10	docker stats、pidstat、strace、node trace、iostat、perf、vmstat	500/500

这组实验控制得比较干净：总请求数一样，消息内容一样，运行环境一样，采集手段也一样。最大的区别只有两个：一边只有 1 个 worker 和 1 个会话，另一边是 10 个 worker 和 10 个会话。

结果先看图



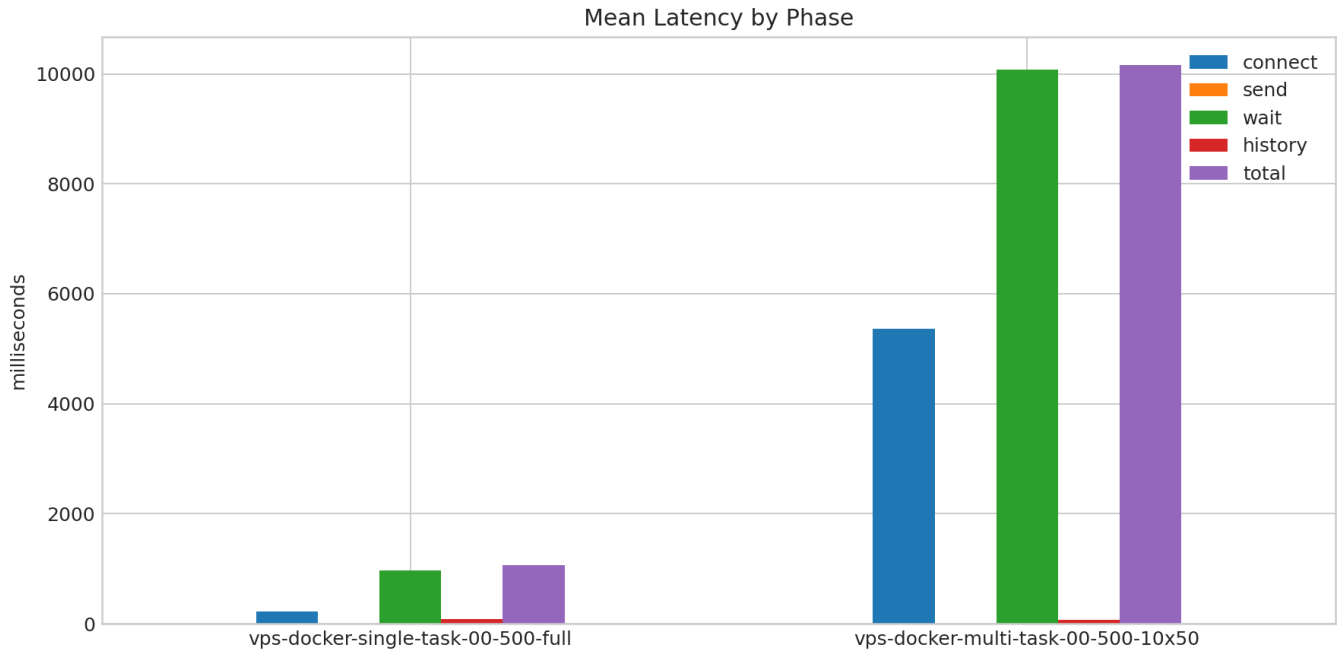
场景	总耗时均值	总耗时 95 分位	等待阶段均值	等待阶段 95 分位	发送均值	历史读取均值	连接阶段均值
单 worker 基线	1061.123ms	1336.504ms	973.591ms	1251.777ms	2.847ms	84.659ms	225.969ms
10 worker 并发	10161.143ms	11308.623ms	10082.269ms	11223.202ms	5.665ms	73.183ms	5358.133ms

这里最重要的观察有两个。

第一，总耗时和等待阶段几乎一起被拉长了，说明问题主要发生在“请求交出去以后，到结果真正回来之前”。

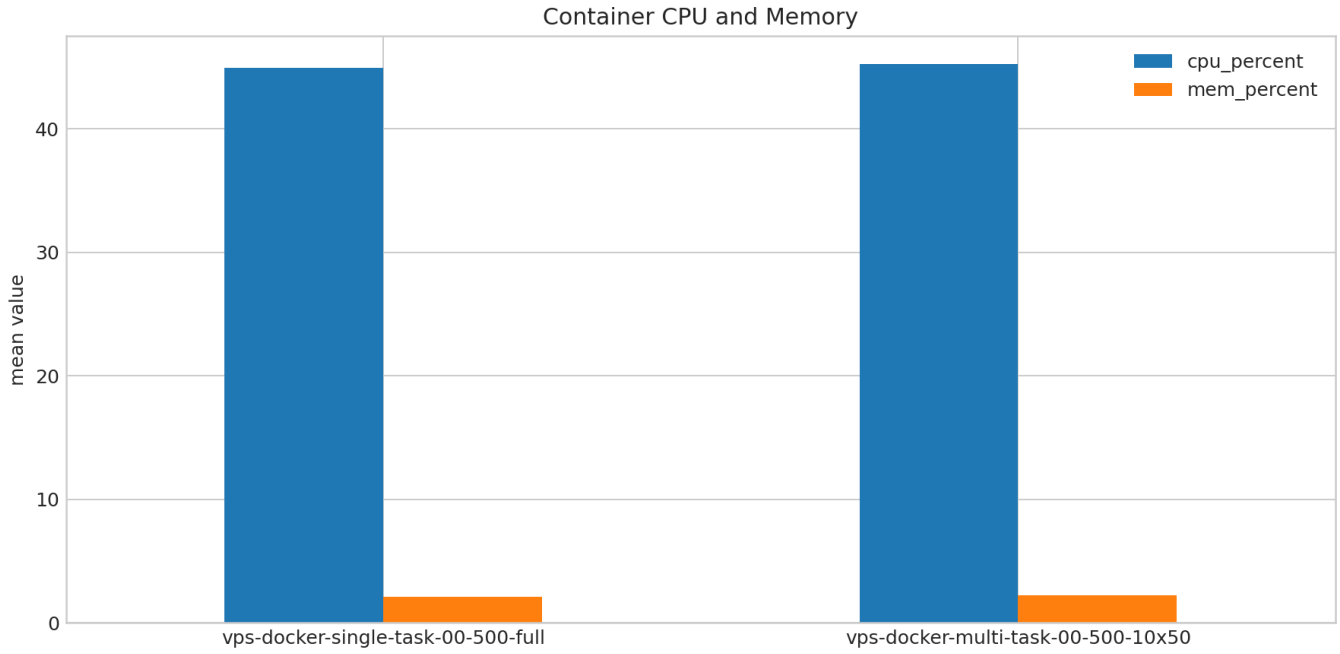
第二，历史读取阶段几乎没变，甚至还略低一点。这意味着瓶颈不在“最后把结果拿回来”，而是在网关内部真正执行和协调的那一段。

需要单独说明的是：连接阶段在多 worker 下也明显升高，但这项指标是按 worker 建连记录的，而且不计入总耗时。所以这份报告的主结论仍然以“总耗时”和“等待阶段”为准。



先排除几个常见但不成立的解释

如果只看“10 倍变慢”这个结果，很容易先把原因归结为 CPU、内存、磁盘，或者误以为是模型推理本身明显变慢。但这组数据并不支持这种解释。



- CPU 平均占用几乎一样：`44.9%` vs `45.3%`。
- 内存平均占用也很接近：`2.07%` vs `2.20%`。
- 磁盘利用率也很接近：`0.390%` vs `0.355%`。
- 磁盘写等待也没有明显恶化：`1.276ms` vs `1.326ms`。
- 回复发回队列几乎没变化：`0.002ms` vs `0.002ms`。

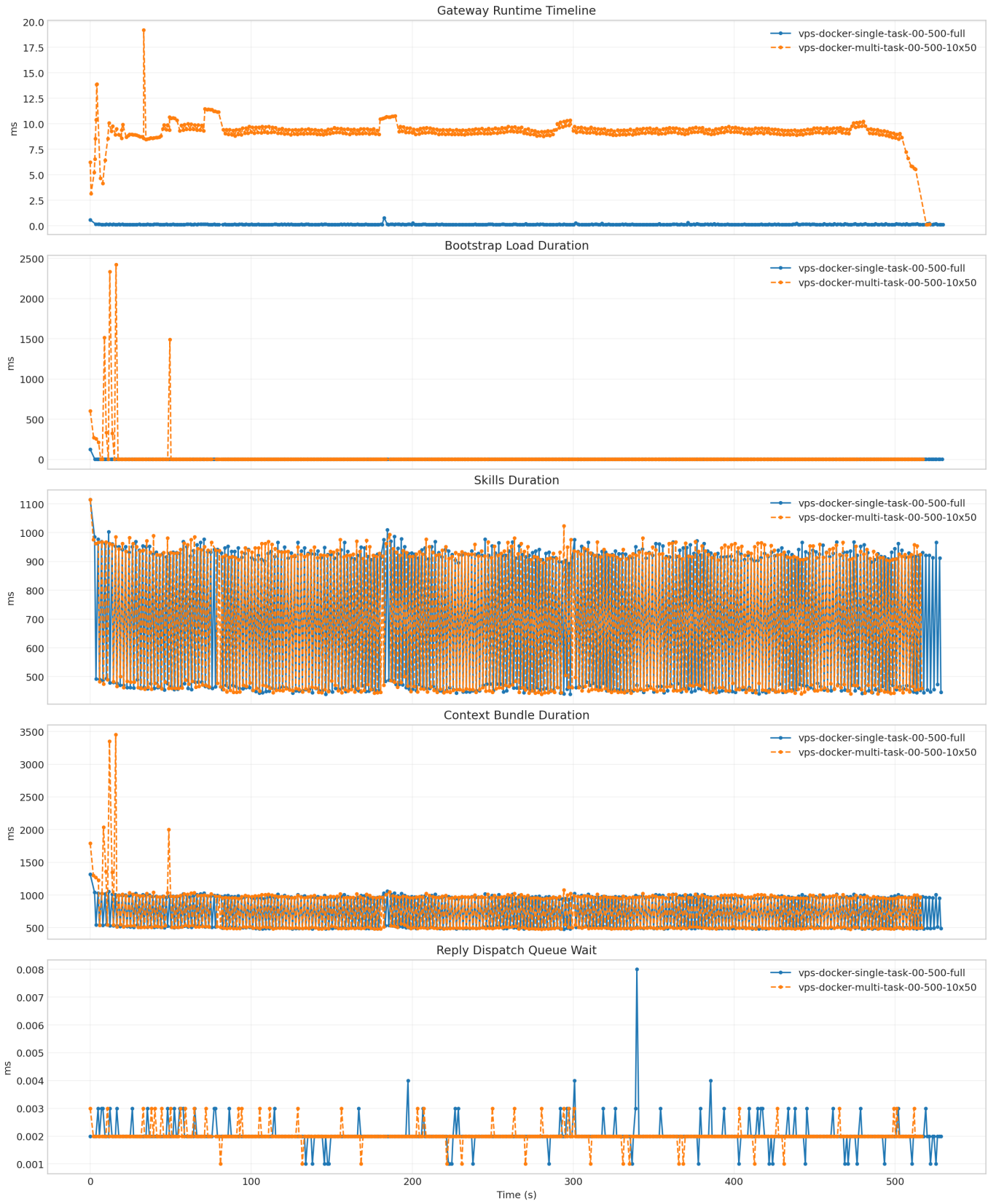
- 技能信息整理几乎没变化：`698.850ms` vs `700.802ms`。
- 上下文总拼装只小幅增加：`742.934ms` vs `764.308ms`。

换句话说，这并不是典型的资源打满问题，也不是模型推理能力本身发生了明显变化。真正的瓶颈在别处。

网关内部到底慢在哪里

从大流程上看，一次请求进入网关后，可以粗略理解成下面几步：

提交请求 -> 等待开始执行 -> 准备基础上下文 -> 整理技能信息 -> 拼装完整上下文 -> 产出回复 -> 发回结果 -> 读取历史



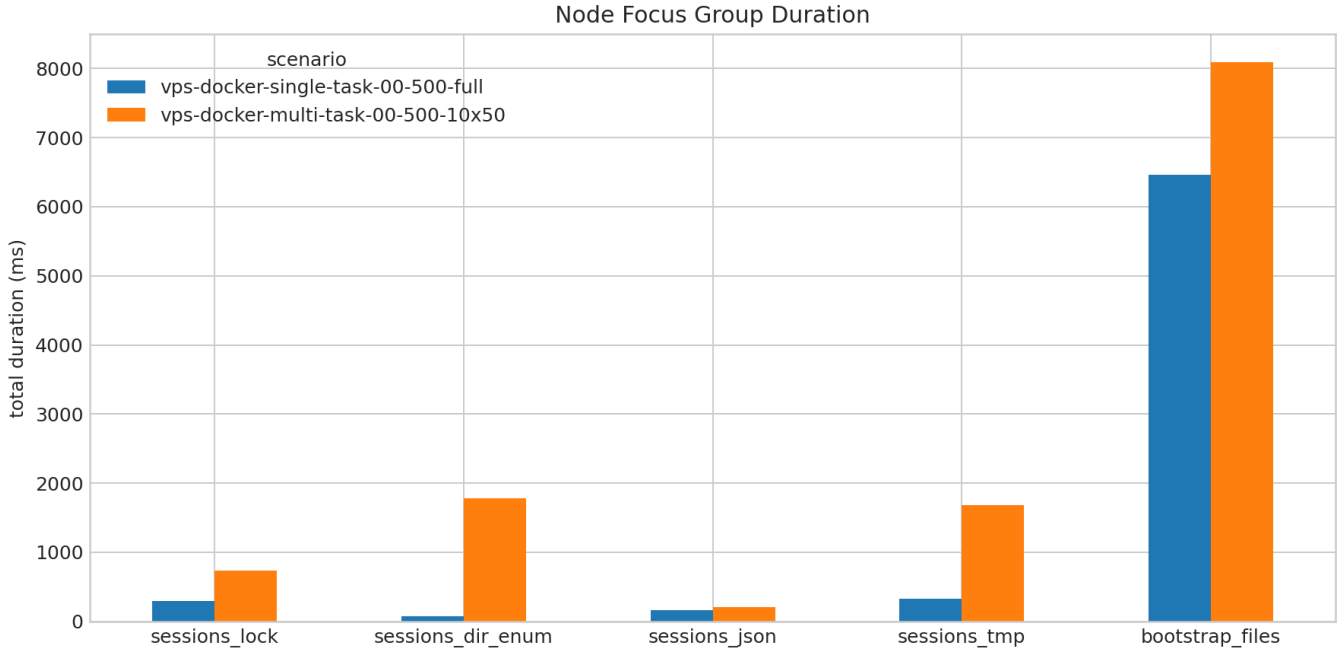
场景	基础上下文加载均值	技能整理均值	上下文总拼装均值	真正开始执行前等待均值	回复队列等待均值
单 worker 基线	2.758ms	698.850ms	742.934ms	0.143ms	0.002ms
10 worker 并发	22.000ms	700.802ms	764.308ms	9.253ms	0.002ms

这张表的意义不在于“哪一项绝对值最大”，而在于帮我们先排除错误方向。

技能整理几乎没变，回复队列几乎没变，上下文总拼装也只是小幅增加。真正能看出变化的，是基础上下文加载，以及“真正开始执行前的等待”。这说明瓶颈并不主要出在实际执行和结果返回阶段，而是集中在执行前的准备、会话协调和准入等待阶段。

真正的根因：多会话把公共链路挤住了

顺着这个方向往下看，真正明显放大的，是会话状态管理这条公共链路。



场景	会话锁累计时长	会话目录扫描累计时长	会话索引文件累计时长	会话临时文件累计时长	基础工作区文件累计时长
单 worker 基线	298.634ms	77.678ms	164.106ms	327.700ms	6459.129ms
10 worker 并发	733.853ms	1781.805ms	210.497ms	1679.839ms	8091.316ms

这里有三个数字需要重点关注。

- 会话目录扫描从 `77.678ms` 增加到 `1781.805ms`。
- 会话临时文件链从 `327.700ms` 增加到 `1679.839ms`。
- 会话锁文件从 `298.634ms` 增加到 `733.853ms`。

这意味着什么？

说明多 worker 并发时，系统不是单纯多做了几次“读文件”这么简单，而是多了很多围绕会话状态的协调动作：要看目录、要碰锁文件、要更新索引、要处理临时文件。它们共用同一条异步文件系统处理通道，互相挤占，就会把“请求已经到了，但真正开始干活之前”的那段时间拉长。

因此，更准确的表述是“会话状态管理链路拥堵”，而不是笼统地说“文件系统慢了”。真正被放大的，是会话相关的公共管理开销。

为什么看起来连“基础上下文准备”也变慢了

这一点特别容易误读。

表面上看，基础上下文加载均值从 `2.758ms` 涨到了 `22.000ms`，好像连最基本的上下文准备都被拖慢了。但把每个会话的第一次请求单独拿出来看，结论就不一样了。

单 worker 场景里，只有 `1` 个会话，所以只付出 `1` 次首次冷启动（cold start）成本。10 worker 并发场景里，有 `10` 个会话，所以会重复付出 `10` 次首次冷启动成本。

最关键的证据是：去掉每个会话的第一条请求后，基础上下文加载均值几乎完全一样。

- 单 worker：`2.511ms`
- 10 worker 并发：`2.492ms`

这说明热态下它并没有真正变慢。它之所以在总均值上看起来更慢，主要是因为会话数从 `1` 个变成了 `10` 个，首次冷启动被重复触发了 `10` 次。

所以，基础上下文准备不是主因。更准确地说，它是“多会话带来更多冷启动”的伴生放大项。

这件事该怎么改

短期可做：

- 尽量复用会话，不要让短任务一上来就把 worker 和 session一一绑定。
- 如果业务允许，优先尝试共享会话，或者至少减少并发时的会话数量。
- 报表里单独统计首次冷启动和热态请求，不要把两者混在一个平均值里。
- 优先优化会话状态管理链路，减少目录扫描、锁文件操作和临时文件往返。

最后一句话

更准确地说，本次性能下降并不是处理能力本身明显减弱，而是并发会话增多后，会话建立与状态管理流程先成为了瓶颈。

补充一句辅助验证：另一组发送`/history head 1`消息的对比实验里，总耗时均值也从 `626.935ms` 增加到 `6088.424ms`，方向一致。